

DEREXI: POLICY PILOT

Cryptographic Fundamentals

Caesar Cipher • XOR Cipher • Mathematics • Security Guarantees

OBJECTIVE

This lab demonstrates two basic cryptographic primitives in Python and explains the mathematical ideas behind each one. The goal is educational: show how plaintext becomes ciphertext, how keys influence the transformation, and why a working cipher is not automatically a secure modern cipher. All sample text is fictional and the lab is isolated from DeRexi production code.

PRIMITIVE 1 • CAESAR CIPHER

CAESAR IMPLEMENTATION

```
CAESAR_SHIFT = 3
shifted = (ord(ch) - base + shift) % 26
return caesar_encrypt(ciphertext, -shift)
```

Plaintext: AURUM CAPITAL BANK POLICY
Shift: 3
Verification: PASS

01 Mathematics

Letters are mapped to 0-25. Encryption uses $E(x)=(x+k) \bmod 26$; decryption uses $D(x)=(x-k) \bmod 26$.

02 Security

Only 25 meaningful shifts exist. Brute force and frequency analysis make Caesar unsuitable for sensitive data.

WORKED EXAMPLE

$A = 0, k = 3 \quad (0 + 3) \bmod 26 = 3$ D. The same modular arithmetic wraps around the alphabet, so Z shifted by 3 becomes C.

CORE CRYPTOGRAPHIC VOCABULARY

The lab explicitly defines plaintext, ciphertext, encryption, decryption, key, algorithm/cipher, and confidentiality. It also applies Kerckhoffs's principle: security should depend on keeping the key secret, not on hiding how the algorithm works.

PAGE 1 RESULT

Caesar encryption/decryption: PASS
Mathematical principles: documented with formulas and worked examples
Security guarantee: VERY WEAK - appropriate as a teaching primitive, not production encryption

DEREXI: POLICY PILOT

XOR, Reversibility & Security Limits

Bitwise logic • Repeating-key weakness • One-time-pad conditions

PRIMITIVE 2 • XOR CIPHER

XOR IMPLEMENTATION

```
XOR_DEMO_KEY = b"DeRexi"
ciphertext = xor_crypt(plaintext_bytes, key)
recovered = xor_crypt(ciphertext, key)
verified = recovered == plaintext_bytes
```

```
P XOR K = C
C XOR K = P
```

01 Reversible logic

Applying XOR with the same key twice restores the original data because matching key bits cancel.

02 Key-reuse risk

A short repeating key creates predictable structure. XOR is an operation; its security depends entirely on key quality and use.

XOR MATHEMATICS

Truth table: $0 \text{ XOR } 0 = 0$, $0 \text{ XOR } 1 = 1$, $1 \text{ XOR } 0 = 1$, $1 \text{ XOR } 1 = 0$. The lab also shows a byte-level worked example and explains why ciphertext is commonly displayed in hexadecimal when the XOR result contains non-printable bytes.

SECURITY GUARANTEES

Caesar: VERY WEAK confidentiality. • Repeating-key XOR: LIMITED / WEAK confidentiality. • One-Time Pad: PERFECT SECRECY only when the key is truly random, secret, as long as the message, and never reused.

EXECUTION / RUBRIC EVIDENCE

```
Cryptography 1 Assignment #1 - Report

Created: security-labs/cryptography/cryptographic_fundamentals.
- no new dependency)

Execution: exit code 0, all 8 headings verified in source and o

Rubric requirements - all passed
```

Requirement	Status
2 basic cryptographic primitives - Caesar Cipher + XOR Cipher	PASSED
Caesar: shift 3. AURUM CAPITAL BANK POLICY → DXJXP FDSLWDDQJ SROJFB → decrypt → verify	PASSED
Caesar math: $E(x) = (x + k) \bmod 26$, $D(x) = (x - k) \bmod 26$, $x/k \bmod 26$ explained, worked example $A=0$, $k=3$, $(0+3) \bmod 26 = 3 \neq 0$	PASSED
Caesar security analysis: substitution only, 25 shifts, trivially brute-forced, preserves letter frequencies, not meaningful security	PASSED
XOR: plaintext=bytes, fictional key DeRexi, ciphertext in hex, XOR again recovers - verify	PASSED